

DhakaSim: Java-to-Python Reimplementation and Interface/Architecture Improvements

Progress Report

Mehzad Galib

July 29, 2026

Abstract

DhakaSim is an event-based microscopic traffic simulator for the non-lane-based, heterogeneous traffic characteristic of cities such as Dhaka. This report documents the work completed so far: the reimplementation of the original Java codebase as a dependency-free Python package validated for numeric fidelity against the Java reference, and a set of improvements to the graphical interface, the output architecture, and the run reporting. Together these turn a faithful but bare research port into a self-documenting, easier-to-use tool.

1 Introduction

Traffic in Dhaka is non-lane-based and heterogeneous: motorised and non-motorised vehicles share the same right-of-way without lane discipline. DhakaSim models this by dividing each road into thin lateral *strips* rather than lanes; a vehicle occupies several strips and may move to any free strip. This report covers two strands of completed work: (i) the Java-to-Python reimplementation that forms the foundation (Section 2); and (ii) the interface, architecture and reporting improvements built on top of it (Sections 3 and 4). Section 5 lists the files changed.

2 The Python Reimplementation of DhakaSim

The simulator was reimplemented from its original Java form as a self-contained Python package (Python 3.10+, standard library only; the GUI uses `tkinter`). The goal was behavioural equivalence with the Java reference rather than idiomatic rewriting.

2.1 Translation approach

The translation follows a one-module-per-class correspondence: each original Java class becomes a Python module of the same responsibility—for example `vehicle.py`, `node.py`, `link.py`, `segment.py`, `strip.py`, `processor.py`, `signal.py`, `pedestrian.py` and `intersection_strip.py`. The package reads its network, demand and parameter files from `input/` and writes results to `statistics/`, mirroring the original layout.

2.2 Numeric-fidelity layer

Python’s default arithmetic differs from Java’s in ways that would shift simulation results, so a dedicated compatibility layer, `javacompat.py`, reproduces the Java semantics that matter:

- `min/max` that propagate NaN (Gipps braking produces NaN routinely, and mishandling it shifts every speed statistic);
- half-up rounding rather than banker’s rounding;
- integer casts that truncate toward zero;

- division by zero and `sqrt` of a negative yielding `Infinity/NaN` instead of raising;
- the sign convention of `%` on negative operands;
- "`% .3f`"-style formatting that rounds the shortest round-tripping decimal half-up;
- a seed-faithful reimplement of `java.util.Random`.

These behaviours are pinned by a regression test, `tests/test_javacompat.py`.

2.3 Validation against the Java reference

With a matched random seed, the Python build reproduces byte-identical `statistics/csv/*.csv` files and identical printed metrics relative to the Java implementation, across all thirteen car-following models, all four lane-changing models, the pedestrian and roadside-object paths (including the per-step accident log), the route/demand generator output, and the drawing geometry written to `trace.txt`. Unseeded, repeated runs agree within statistical noise.

2.4 Testing and execution

The build was exercised end-to-end in headless mode: it completes a run, prints the summary metrics, and produces the full set of per-type and per-route output files. The numeric regression test passes, confirming the Java-compatible arithmetic has not drifted.

3 GUI and Usability Improvements

A set of changes make the interface understandable to first-time users without reading the source.

3.1 Self-documenting option panel

Every field on the “Start Simulation” screen now carries its unit in the label and a plain-language explanation beside it—for example, the simulation end time in seconds, the simulation speed in milliseconds per frame, strip width in metres with the non-lane explanation, and maximum speed in km/h—along with a note of what the random seed and trace mode do. A subtitle explains that a report opens automatically when the run ends.

3.2 Named nodes

Network nodes can now be labelled by name rather than by numeric id. Names are read from an optional `input/node_names.txt` file (one `id name` pair per line), so any network can supply its own; nodes without a name fall back to their id. A latent bug was fixed in passing: junction labels, which had been drawn at the coordinate origin because junction centres are stored as `(0,0)`, are now placed at the true junction position, recovered as the mean of the link endpoints that meet there.

3.3 Units in terminal output

The summary metrics printed at the end of a run now carry units: speeds are reported in km/h and waiting times in seconds, matching the report.

3.4 Per-type vehicle colours

Vehicles were previously drawn in random colours, making them indistinguishable by type. A fixed colour per type is now defined once and used by both the animation and the report, with cars, buses and trucks sharing red, purple and grey families respectively and CNGs drawn green. The original random draw is preserved so runs remain seed-identical.

```

VEHICLE_TYPE_COLORS = (
    (0,160,160), (230,130,20), (140,90,40), (40,110,220), # bicycle..
    motorbike
    (210,50,50), (170,30,60), (240,105,105), # car x3
    (30,160,70), (140,60,190), (200,70,170), # CNG, bus x2
    (100,100,100), (55,55,55), (0,0,0), # truck x2, pedestrian
)

```

3.5 Collapsible in-GUI legend

A small legend panel on the right of the simulation window maps each vehicle colour to its type, drawn from the same colour table as the vehicles. It is collapsible via a `-/+` toggle so it can be tucked away on narrow screens.

4 Architecture and Reporting Improvements

4.1 Automated HTML report

A new module, `report.py`, writes a self-contained, dependency-free HTML report at the end of every run. It presents headline metric cards (overall, motorised and non-motorised speed and waiting), a “what was simulated” configuration table, a per-vehicle-type results table (generated, trips completed, speed, waiting %, trip time, fuel), two SVG bar charts (vehicles generated by type and average speed by type), a colour legend, and a glossary defining every term. In the GUI, a dialog offers to open the report when the run finishes. The summary values are derived from the in-memory statistics rather than by re-reading files, so the report is independent of the CSV outputs.

4.2 Reorganised statistics output

The output layout was cleaned up so that the `statistics/` folder holds only the human-readable reports. Each run’s report is written with a time-stamped filename, `report_YYYYMMDD_HHMMSS.html`, so successive runs are retained like log files rather than overwriting one another. The raw numeric CSVs (per-type, per-route, flow and the per-event accident log) are routed into a `statistics/csv/` sub-folder, keeping the folder root uncluttered while preserving the full numeric record. This routing is centralised, so every CSV writer is redirected in one place.

4.3 Offline visualisation

Because the interactive `tkinter` window must run on the user’s own machine, an offline renderer was produced that reuses the simulator’s own vehicle-drawing geometry to rasterise frames into a still image and an animated preview. This makes it possible to inspect a run’s behaviour without a live display, and it shares the per-type colour scheme so the output matches the in-GUI legend.

5 Summary of Changes

Table 1 lists the source files added or modified in this work.

6 Conclusion

The simulator now stands as a validated Python reimplementaion of DhakaSim that is considerably easier to understand and operate: named nodes, unit-labelled output, type-coloured vehicles and an in-GUI legend on the interface side, and an automatic, self-explaining per-run HTML report plus a tidy, time-stamped output layout on the architecture side. The numeric-fidelity guarantees of the original

File	Change
dhakasim/report.py	New: HTML report generator (metric cards, tables, SVG charts, colour legend, glossary, time-stamped files)
dhakasim/gui.py	Annotated option panel; named nodes with corrected label placement; report dialog on finish; collapsible colour legend
dhakasim/processor.py	Per-type vehicle colours; node-name loading; report hook; km/h units in summary output; CSVs routed to <code>statistics/csv/</code>
dhakasim/constants.py	Fixed per-type vehicle-colour palette
dhakasim/parameters.py	Storage for optional node names
dhakasim/vehicle.py	Accident log written under <code>statistics/csv/</code>
input/node_names.txt	New: optional friendly node names
README.md	Documentation updated for the new output layout, node names and units

Table 1: Source files added or modified.

port are preserved throughout—all colour and reporting changes leave the random stream and computed statistics unchanged.

References

- [1] H. Dev *et al.*, “DhakaSim: A Tool for Simulating Traffic Mobility in Dhaka City,” *MoHCI*, 2014.
- [2] M. M. Mushfiq *et al.*, ““To Lane or Not to Lane?”—Comparing On-Road Experiences in Developing and Developed Countries Using a New Simulator RoadBird,” *IEEE Trans. Intelligent Transportation Systems*, vol. 25, no. 8, pp. 8486–8498, 2024.